INF1022 - Analisadores Léxicos e Sintáticos

JavaCC

Edward Hermann Haeusler Fernando Pinto

PUC-Rio

Departamento de Informática

JavaCC

JavaCC tool

Originally developed at Sun Microsystems Inc. by Sreeni Viswanadha and Sriram Sankar.

JavaCC

JavaCC tool

Originally developed at Sun Microsystems Inc. by Sreeni Viswanadha and Sriram Sankar.

Parser Generator

A parser generator is a tool that reads a grammar specification and converts it to a Java program that can recognize matches to the grammar.

Features (I) - Site Community JavaCC [1]

Top-Down

JavaCC generates top-down (recursive descent) parsers as opposed to bottom-up parsers generated by YACC-like tools.

Features (I) - Site Community JavaCC [1]

Top-Down

JavaCC generates top-down (recursive descent) parsers as opposed to bottom-up parsers generated by YACC-like tools.

LL(1) parser

By default, JavaCC generates an LL(1) parser. However, there may be portions of grammar that are not LL(1).

Features (I) - Site Community JavaCC [1]

Top-Down

JavaCC generates top-down (recursive descent) parsers as opposed to bottom-up parsers generated by YACC-like tools.

LL(1) parser

By default, JavaCC generates an LL(1) parser. However, there may be portions of grammar that are not LL(1).

Portability

JavaCC generates parsers that are 100% pure Java, so there is no runtime dependency on JavaCC and no special porting effort required to run on different machine platforms.

Features (I) - Site Community JavaCC [1]

Top-Down

JavaCC generates top-down (recursive descent) parsers as opposed to bottom-up parsers generated by YACC-like tools.

LL(1) parser

By default, JavaCC generates an LL(1) parser. However, there may be portions of grammar that are not LL(1).

Portability

JavaCC generates parsers that are 100% pure Java, so there is no runtime dependency on JavaCC and no special porting effort required to run on different machine platforms.

Extended BNF

JavaCC allows extended BNF specifications - such as (A)*, (A)+ etc - within the lexical and the grammar specifications.

Features (II) - Site Community JavaCC [1]

Readability

The lexical specifications (such as regular expressions, strings) and the grammar specifications (the BNF) are both written together in the same file.

Features (II) - Site Community JavaCC [1]

Readability

The lexical specifications (such as regular expressions, strings) and the grammar specifications (the BNF) are both written together in the same file.

JJTree

JavaCC comes with JJTree, an extremely powerful tree building pre-processor.

Features (II) - Site Community JavaCC [1]

Readability

The lexical specifications (such as regular expressions, strings) and the grammar specifications (the BNF) are both written together in the same file.

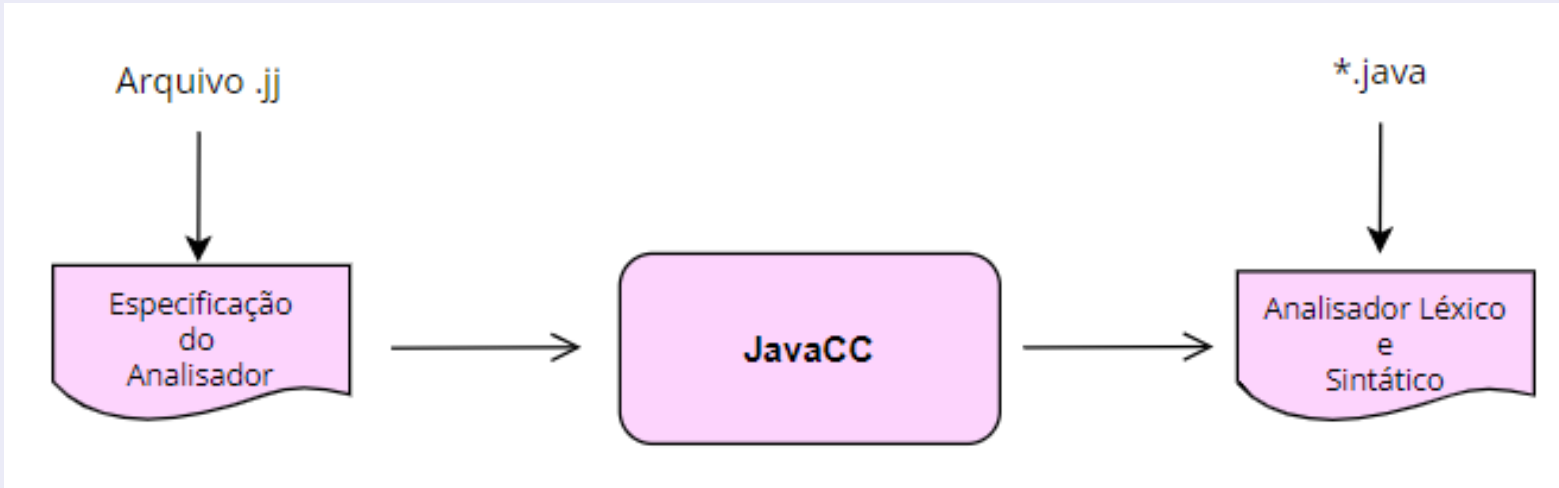
JJTree

JavaCC comes with JJTree, an extremely powerful tree building pre-processor.

JJDoc

JavaCC also includes JJDoc, a tool that converts grammar files to documentation files, optionally in HTML.

Process


Arquivo .jj
*.java
Especificação
do
Analisador
JavaCC
Analisador Léxico
e
Sintático

Running JavaCC

Command line operation

```
$ javacc file.jj
$ javac *.java
$ java main<main.class> algo.x
```

Running JavaCC

Command line operation

```
$ javacc file.jj
$ javac *.java
$ java main<main.class> algo.x
```

Command line operation

```
$ javacc simple.jj
Java Compiler Compiler Version 4.2 (Parser Generator)
(type "javacc" with no arguments for help)
Reading from file simple.jj . . .
File "TokenMgrError.java" does not exist.  Will create one.
File "ParseException.java" does not exist.  Will create one.
File "Token.java" does not exist.  Will create one.
File "SimpleCharStream.java" does not exist.  Will create one.
Parser generated successfully.
$ ls *.java
ParseException.java
SimpleCharStream.java
SimpleParser.java
SimpleParserConstants.java
SimpleParserTokenManager.java
Token.java
TokenMgrError.java
$ javac *.java
$ java SimpleParser
Found an 'a'!
```

Regular Expressions [2]

Name	Example	Description
Literal	"a"	Matches only an "a"
Character class	["a","b","c"]	Either an "a", a "b", or a "c"
Ranged character class	["a"-"z"]	All lowercase letters
Negation	~["a"]	Anything single character other than an "a"
Repetition	("a"){4}	Matches 4 "a"s
Repetition ranges	("a"){2,4}	Matches at least 2 but not more than 4 "a"s
One or more items	("a")+	At least one "a"
Zero or one items	("a")?	Either zero or one "a"
Zero or more items	("a")*	Any number of "a"s (including none)

Token Sintaxe JavaCC

```
TOKEN :
{
        <TOKEN_ID1 : RE1>
      | <TOKEN_ID2 : RE2>
      | <TOKEN_ID3 : RE3>
      | ...
      | <TOKEN_IDn : REn-1>
      | <TOKEN_IDn : REn>
}
```

Character classes

Simple

```
1  TOKEN : 
2  { 
3    <A_or_B_or_C : ["a", "b", "c"]> 
4  }
```

Character classes

Simple

```
1  TOKEN : 
2  { 
3    <A_or_B_or_C : ["a", "b", "c"]> 
4  }
```

Ranged

```
1  TOKEN : 
2  { 
3    <A_to_C : ["a"-"c"]> 
4  }
```

Character classes

Simple

```
1 TOKEN : 
2 {
3   <A_or_B_or_C : ["a", "b", "c"]>
4 }
```

Ranged

```
1 TOKEN : 
2 {
3   <A_to_C : ["a"-"c"]>
4 }
```

Alternation and ranges

```
1 TOKEN : 
2 {
3   <A_to_C_or_X_to_Z : ["a"-"c"] | ["x"-"z"]>
4 }
```

Character classes

Mixing character classes and ranged character classes

```
1 TOKEN : 
2 {
3   <A_or_C_or_X_to_Z : ["a","c", "x"-"z"]>  
4 }
```

Negation

Simple negation grammar

```
1  TOKEN : 
2  {
3    <ENYTHING_BUT_A : ~["a"]>
4  }
```

Negation

Simple negation grammar

```
1  TOKEN : 
2  {
3    <ENYTHING_BUT_A : ~["a"]> 
4  }
```

Negation range grammar

```
1  TOKEN : 
2  {
3    <ENYTHING_BUT_A_to_C : ~["a"-"c"]> 
4  }
```

?

```
1  TOKEN · :
2  {
3  · · <TKN1 · : · "a" · ["a"-"c"]> · ·
4  }
```

?

```
1  TOKEN · :
2  {
3  · · <TKN1 · : · "a" · ["a"-"c"]> · ·
4  }
```

```
1  TOKEN · :
2  {
3  · · <TKN2: · (["+","-"])? · (["0"-"9"])*>
4  }
```

Bibliography I

COMMUNITY., J.
Javacc.
https://javacc.github.io/javacc/#download, accessed May, 2022).

COPELAND, T.
Generating Parsers with JavaCC.
2009.